SCHEMA DESIGN — INTERNAL ENGINEERING DOC

Astra AI persistence model

Full entity design for the tables replacing today's in-memory user store, stateless-only JWTs, `usage_data.json`, and browser-localStorage favorites — targeting the Supabase Postgres instance the team is standing up.

11 entities **Target** Supabase Postgres **Normal form** verified to 3NF **Vectors** remain in ChromaDB

---

01 — OVERVIEW

What this schema has to hold

Five things happen in the running app today, each currently backed by something that doesn't survive a restart except the last:

1. **auth** — a user signs in; a JWT is issued. Today this is stateless: no record of the login exists anywhere, so it can't be revoked, listed, or counted reliably.
2. **chat** — a question is asked, retrieval runs, an answer streams back citing 1–5 source chunks. Today this list lives only in the backend's in-memory `CHAT_HISTORY`, wiped on every restart.
3. **favorite** — a user pins up to 5 conversations and can rename them. Today this is browser localStorage only — it survives backend restarts, but not a cleared cache or a new device.
4. **ingest** — an admin uploads an SOP or technical manual; it's chunked and embedded. Chunk vectors live in ChromaDB; each chunk currently repeats its parent document's category/client/type as its own metadata.
5. **meter** — every model call is checked and counted against a per-user, per-model daily budget across 4 Groq models spent in priority order. Today this is a single JSON file on disk.

Eleven tables

Grouped by the flow each one serves. Every PK is a real primary key; every FK a foreign key; UK a unique constraint.

users

one row per person who can sign in

COLUMN	TYPE	KEY	NOTES
id	uuid	PK	—
email	text	UK	login identifier
name	text		display name
hashed_password	text		bcrypt hash
role	text		manager · lead_analyst · solar_analyst
created_at	timestamptz		—

sessions

one row per login — makes JWT auth revocable and countable

COLUMN	TYPE	KEY	NOTES
id	uuid	PK	equals the JWT's `jti` claim
user_id	uuid	FK	→ users.id
created_at	timestamptz		login time
expires_at	timestamptz		token expiry
last_active_at	timestamptz		bumped per request
revoked	boolean		default false
ip_address	inet		nullable
user_agent	text		nullable

conversations

one row per chat thread

COLUMN	TYPE	KEY	NOTES
id	uuid	PK	—
user_id	uuid	FK	→ users.id
title	text		auto-generated, or renamed by owner
created_at	timestamptz		drives the 10-chat FIFO queue
updated_at	timestamptz		drives "recent" sort order

messages

one row per turn — a question or an answer

COLUMN	TYPE	KEY	NOTES
id	uuid	PK	—
conversation_id	uuid	FK	→ conversations.id
role	text		user · assistant
content	text		question or streamed answer
model_key	text	FK	→ models.model_key, null on user turns / FAQ hits
is_faq_hit	boolean		answered free from cache
created_at	timestamptz		—

message_sources

one row per document an answer cites — see 1NF, below

COLUMN	TYPE	KEY	NOTES
id	uuid	PK	—
message_id	uuid	FK	→ messages.id
document_id	uuid	FK	→ documents.id
chunk_index	int		nullable
similarity_score	real		nullable, cosine distance

favorites

one row per pinned conversation — permanent, max 5/user, owner-only delete

COLUMN	TYPE	KEY	NOTES
user_id	uuid	PK FK	→ users.id
conversation_id	uuid	PK FK	→ conversations.id
favorited_at	timestamptz		the "max 5" rule is enforced in the app layer, by counting rows

documents

one row per ingested file — see 3NF, below

COLUMN	TYPE	KEY	NOTES
id	uuid	PK	—
filename	text		—
category	text		Case Creation, Alerts, PV Technical, BESS...
client_name	text		Clean Leaf, Technical Library...
file_type	text		.pdf .docx .xlsx ...
total_chunks	int		count in ChromaDB
uploaded_by	uuid	FK	→ users.id, nullable
uploaded_at	timestamptz		—

faq_entries

one row per cached Q&A — free, instant answers

COLUMN	TYPE	KEY	NOTES
id	uuid	PK	—
question	text		embedding lives in ChromaDB, keyed by this id
answer	text		—
source_document_id	uuid	FK	→ documents.id, nullable
created_at	timestamptz		—

models

lookup table — the 4 Groq models and their budgets, currently a hardcoded Python dict

COLUMN	TYPE	KEY	NOTES
model_key	text	PK	gpt120 · llama70 · gpt20 · llama8
label	text		"GPT-OSS 120B"...
groq_model_id	text		the literal Groq API model string
user_daily_tokens	int		per-user budget
user_daily_requests	int		per-user budget
global_daily_tokens	int		Groq's team-wide TPD cap
global_daily_requests	int		Groq's team-wide RPD cap
spend_priority	int		1 = spent first

usage_model_daily

one row per user, per model, per day — the budget ledger

COLUMN	TYPE	KEY	NOTES
user_id	uuid	PK FK	→ users.id
model_key	text	PK FK	→ models.model_key
usage_date	date	PK	—
tokens_used	int		default 0, exact count from Groq's response
requests_used	int		default 0

usage_daily_summary

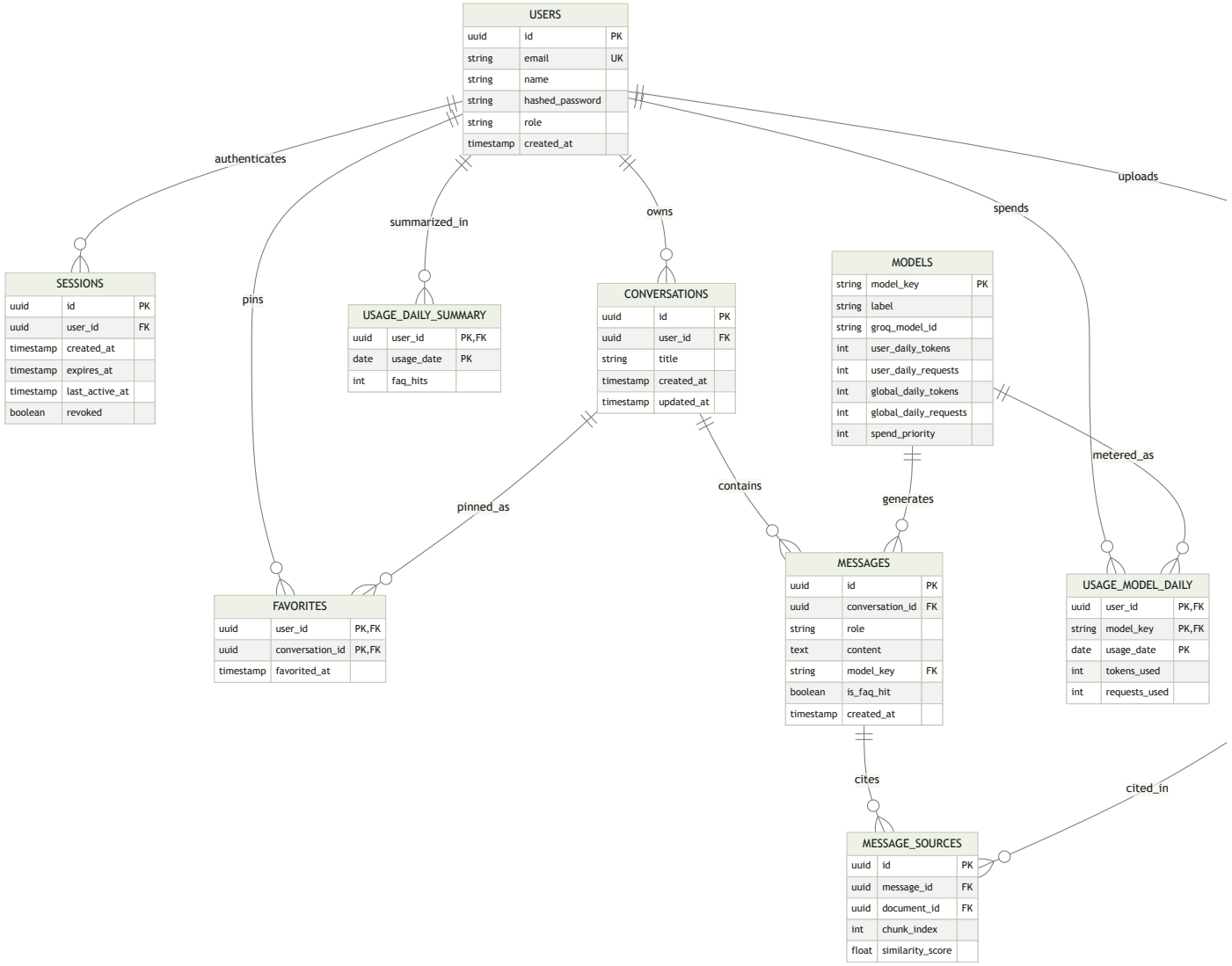
one row per user, per day — see 2NF, below

COLUMN	TYPE	KEY	NOTES
user_id	uuid	PK FK	→ users.id
usage_date	date	PK	—
faq_hits	int		default 0 — the only fact at this grain

03 — ENTITY-RELATIONSHIP DIAGRAM

How the eleven tables connect

Full columns are in the data dictionary above; this shows shape and cardinality only.



Composite keys show PK, FK where a column is simultaneously part of the primary key and a foreign key.

04 — NORMALIZATION

1NF → 2NF → 3NF, checked against the real data

Each level below only matters because of something the current implementation actually does. Three concrete redesigns came out of the check.

1NF Every column holds one atomic value — no repeating groups

An answer routinely cites 3–4 source documents at once. Storing that list as one column on `messages` — a JSON array or a joined string — would put a multi-valued, repeating group inside a single cell: a direct 1NF violation.

REJECTED

`messages.sources` = `["Ops Review.docx", "Reactive Case creation.docx"]`

APPLIED

One row per citation in `message_sources` — a message with 4 sources is 4 rows, each atomic

- All 11 tables compliant

2NF No attribute depends on only part of a composite key

Three tables have composite primary keys: `favorites`, `usage_model_daily`, and `usage_daily_summary`. The risk is a column that only cares about *part* of the key it sits under. The original `usage_data.json` nested `faq_hits` and `logins` at the user level, alongside a per-model breakdown — conceptually two different grains sharing one record.

REJECTED

One table keyed on `(user_id, model_key, date)` holding `faq_hits` — a fact that depends only on `(user_id, date)`, not on `model_key`

APPLIED

Split by grain: `usage_model_daily` (per model) and `usage_daily_summary` (per user/day, model-independent)

- All 11 tables compliant

3NF No non-key attribute depends on another non-key attribute

Today, every single chunk's metadata in ChromaDB repeats its parent document's `category`, `client_name`, and `file_type` — all 2,547 chunks carry their own copy. Ported directly into a relational chunk-level table, that's a transitive dependency: `chunk_id → filename → category`.

REJECTED

Chunk-level table repeating `category` / `client_name` / `file_type` per chunk

APPLIED

Those four attributes live exactly once, in `documents`; chunks (and citations) reference it by `document_id`

A second, subtler case: the original design stored `logins` and `last_active` as standalone user fields. Once a `sessions` table exists — one row per login — both are just queries over it (`COUNT(*)` and `MAX(last_active_at)`), not facts that need their own column. Storing both would let the two representations drift apart.

REJECTED

`users.logins`, `users.last_active` as stored, separately-updated columns

APPLIED

Dropped — derived at read time from `sessions`; only `faq_hits` stays stored, since it isn't derivable from anything else

Watch item, not a violation today: `users.role` is a bare label with no attributes of its own, so it isn't a 3NF problem yet. The moment the role itself needs describable data (a permission set, a max-favorites override), pull it into a `roles` lookup table before that data lands on `users` — otherwise `user_id → role → role_description` becomes transitive.

- All 11 tables compliant, one watch item flagged

05 — MIGRATION MAP

What each table is replacing

Vector embeddings — document chunks and FAQ question vectors — stay in ChromaDB; only their descriptive metadata moves into Postgres.

TODAY	BECOMES	WHY IT MOVES
in-memory USERS dict	users	survives a restart at all
stateless-only JWT	sessions	revocable, listable, countable logins
in-memory CHAT_HISTORY list	conversations + messages	survives restarts; powers the admin team-wide view

TODAY	BECOMES	WHY IT MOVES
browser localStorage conversations	conversations + messages	syncs across devices, not just one browser
localStorage favorite flag	favorites	server-enforced 5-max, survives a cleared cache
`usage_data.json`	usage_model_daily + usage_daily_summary	survives Render's free-tier restarts
hardcoded `MODELS` dict	models	single source of truth, editable without a redeploy
ChromaDB per-chunk metadata	documents	removes the repeated-per-chunk redundancy (3NF)
ChromaDB FAQ collection metadata	faq_entries	question embeddings stay in Chroma; answer text moves to Postgres

---

Astra AI SolarOps · schema proposal for the Supabase migration · vector data (chunk and FAQ-question embeddings) intentionally stays in ChromaDB — this document covers the relational side only.